

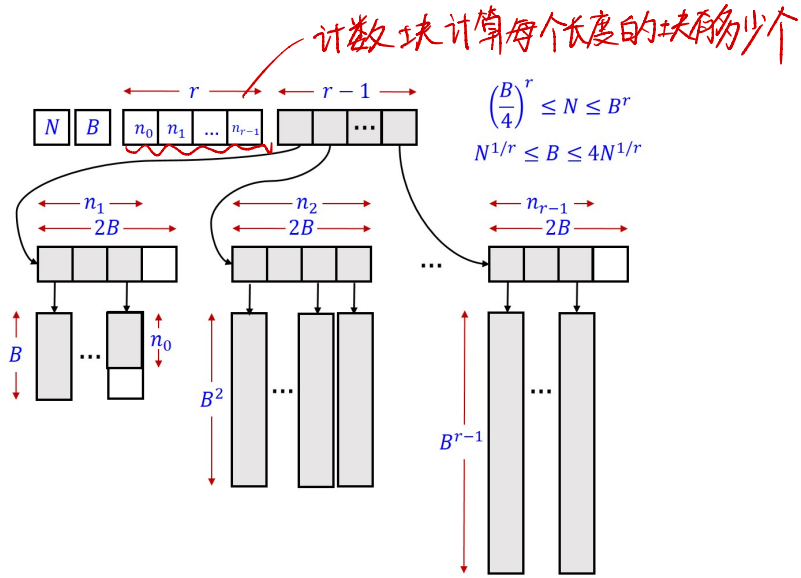


Figure 4: An $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation, for every $r \geq 2$.

are deallocated. A new block of size B is now allocated and the new item is placed in it. Note that this mimics the operation of a base- B counter. // 正常 grow 操作

When $N = B^r$, the value of B is doubled and the whole data structure is rebuilt. (We can delay the rebuilding until the data structure is completely full, i.e., $N = B^r + B^{r-1} + \dots + 1 = \frac{B^{r+1}-1}{B-1}$, but nothing much is gained by it. For simplicity, we rebuild when $N = B^r$.) // 彻底重构

The data structure maintains $r - 1$ index blocks, one for each possible block size. The total space used to store an array containing N items is at most $N + (2r - 1) + 2(r - 1)B + B = N + O(rN^{1/r})$. The maximum amount of extra space used by the implementation is $O(N^{1-1/r})$, when a new block of size B^{r-1} is allocated.

It is easy to see that each item can be accessed in $O(r)$ worst-case time. With some more care, this can be reduced to $O(1)$ worst-case time, as we show in Section 6.3.

The amortized cost of a grow operation is $O(r)$ since each item is first placed in a block of size B and eventually moved to blocks of size B^i , for $i = 2, \dots, r - 1$. A complete rebuilding adds only a constant to the amortized cost of a grow operation since the total number of items in the data structure increases by some constant factor between two rebuild operations. $\frac{B^{r+1}-B^r}{B^{r+1}-B^r} < \frac{B^r}{B^{r+1}-B^r} = \frac{1}{B-1}$ // 证明

Allowing shrink operations while maintaining an $O(r)$ amortized cost per grow and shrink operation is not difficult. A standard solution is to use a *redundant* base- B counter. We allow up to $2B$ blocks of each size, not just B . When there are $2B$ full blocks of a given size B^i , for $1 \leq i < r - 1$, a new block of size B^{i+1} is allocated, The first B blocks of size B^i are copied into it and then deallocated. If a shrink operation occurs when there are no blocks of size B , a block of size B^i , for the smallest possible i , is split into $B - 1$ blocks of each size $B^{i-1}, \dots, B^2$ and B blocks of size B . We show in Section 6.2 that the amortized cost of both grow and shrink operations is again $O(r)$. A similar redundant base- B counter is used in a different setting by Kaplan et al. [10]. // 删除方式

6.1 Implementation details.

Pseudocode for grow and shrink operations is given in Figure 5. As in Figure 4, we assume that n_i , for $i \in [r - 1]$, is the number of data blocks of size B^i . We denote these blocks by $A[i][0], \dots, A[i][n_i - 1]$. If $n_1 > 0$, we let n_0 be the number of items in $A[1][n_1 - 1]$, the last data block of size B . With this notation, $A[i]$, for $1 \leq i \leq r - 1$, is the i -th index block, with size is $2B$, and A is the block of pointers to the index blocks, with size r . Indices in each block start from 0. (For convenience,

从 $i=1$ 起加

Grow(a):

```

if  $N = B^r$  :
    Rebuild( $2B$ )
else if  $n_1 = 2B$  and  $n_0 = B$  :
    Combine-Blocks()
else if  $n_1 = 0$  or  $n_0 = B$  :
     $A[1][n_1] \leftarrow \text{Allocate}(B)$ 
     $n_1 \leftarrow n_1 + 1$ 
     $n_0 \leftarrow 0$ 
 $A[1][n_1 - 1][n_0] \leftarrow a$ 
 $n_0 \leftarrow n_0 + 1$ 
 $N \leftarrow N + 1$ 

```

Combine-Blocks():

```

 $k \leftarrow \min\{i \in [r-1] \mid n_i < 2B\}$ 
if  $k = \infty$  : error
for  $i \leftarrow k-1$  downto 1 : 保证数据不在复制前被销毁
     $A[i+1][n_{i+1}] \leftarrow \text{Allocate}(B^{i+1})$ 
    for  $j \leftarrow 0$  to  $B-1$  :
         $\text{Copy}(A[i][j], 0, A[i+1][n_{i+1}], jB^i, B^i)$ 
         $\text{Deallocate}(A[i][j])$ 
         $A[i][j] \leftarrow A[i][j+B]$  // Shift indices.
     $n_i \leftarrow B$ 
     $n_{i+1} \leftarrow n_{i+1} + 1$ 

```

Shrink():

```

if  $N = (B/4)^r$  :
    Rebuild( $B/2$ )
else if  $n_1 = 0$  : 保证 grow 时
    Split-Blocks() 不会出现  $i=2$  部分空闲块情况
 $n_0 \leftarrow n_0 - 1$ 
 $N \leftarrow N - 1$ 
if  $n_0 = 0$  :
     $\text{Deallocate}(A[1][n_1 - 1])$ 
     $n_0 \leftarrow B$ 
     $n_1 \leftarrow n_1 - 1$ 

```

Split-Blocks():

```

 $k \leftarrow \min\{i \in [r-1] \mid n_i > 0\}$ 
if  $k = \infty$  : error
for  $i \leftarrow k-1$  downto 1 :
     $n_{i+1} \leftarrow n_{i+1} - 1$ 
    for  $j \leftarrow 0$  to  $B-1$  :
         $A[i][j] \leftarrow \text{Allocate}(B^i)$ 
         $\text{Copy}(A[i+1][n_{i+1}], jB^i, A[i][j], 0, B^i)$ 
         $\text{Deallocate}(A[i+1][n_{i+1}])$ 

```

Figure 5: Pseudocode of grow and shrink operations of the $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation.

we leave the cell $A[0]$ empty.) The numbers $n_0, n_1, \dots, n_{r-1}$ are kept in a block n of size r . (For simplicity we write n_i instead of $n[i]$.) The blocks A and $A[i]$, for $1 \leq i \leq r-1$, are allocated and deallocated only during rebuild operations. // 图4解释和符号

The pseudocode assumes the existence of a procedure $\text{Allocate}(\ell)$ that allocates a new block of size ℓ and returns a pointer to the newly allocated block, and a procedure $\text{Deallocate}(X)$ that deallocates a previously allocated block X . It also assumes that $\text{Copy}(X, x, Y, y, \ell)$ copies the items of $X[x..x+\ell-1]$ to $Y[y..y+\ell-1]$, where $X[x..x+\ell-1]$ is shorthand for $X[x], X[x+1], \dots, X[x+\ell-1]$. (In other words, it does the assignments $Y[y+i] \leftarrow X[x+i]$, for $i = 0, 1, \dots, \ell-1$.) A procedure $\text{Rebuild}(B')$, not shown, rebuilds the data structure with the new parameter B' , usually either $2B$ or $B/2$. // 调用的函数

Grow(a) works as follows. If $N = B^r$, the data structure is rebuilt with $B \leftarrow 2B$. If the first level is completely full, i.e., $n_1 = 2B$ and $n_0 = B$, *Combine-Blocks()* is called. This procedure finds the smallest index $k \in [r-1]$ for which $n_k < 2B$. (Such an index must exist.) For $i \leftarrow k-1, k-2, \dots, 1$, it combines B blocks of size B^i into a new block of size B^{i+1} , allocating and deallocating blocks as needed. More specifically, to maintain order, the first B blocks of size B^i , i.e., $A[i][0], \dots, A[i][B-1]$, are combined to form a new last block $A[i+1][n_{i+1}]$ of size B^{i+1} . The blocks $A[i][B], \dots, A[i][2B-1]$ become blocks $A[i][0], \dots, A[i][B-1]$. (This only involves pointer changes. No items are moved. It is possible to avoid these pointer changes by considering $A[i]$ to be a cyclic array.) Finally, the number of blocks of size B^i is set to B and the number of blocks of size B^{i+1} is increased by one,

i.e., $n_i \leftarrow B$ and $n_{i+1} \leftarrow n_{i+1} + 1$.

If *Rebuild* or *Combine-Blocks* are not called, *Grow(a)* checks whether there is a vacant position in a block of size B . If not, i.e., if $n_1 = 0$ (no blocks of size B), or $n_1 > 0$ but $n_0 = B$ (all blocks of size B are full), a new empty block of size B , $A[1][n_1]$, is allocated, n_1 is incremented and n_0 is set to 0. Finally, in all cases, the new item a is placed in the first vacant position of the empty or partially filled B block, namely, $A[1][n_1 - 1][n_0] \leftarrow a$ and n_0 and N are incremented.

Shrink is similar. If $N = (B/4)^r$, the data structure is rebuilt with $B \leftarrow B/2$. Otherwise, if $n_1 = 0$, i.e., there are no blocks of size B , *Split-Blocks* finds the smallest $k \in [r - 1]$ such that $n_k > 0$. The last block of size B^k is split into $B - 1$ blocks of each size $B^{k-1}, \dots, B^2$ and into B blocks of size B . Finally, in all cases, the last item in the last block of size B is discarded and n_0 and N are decremented. If $n_0 = 0$, the empty block of size B is deallocated, n_1 is decremented and n_1 is set to B . (It is actually better to delay the deallocation of the last block of size B , as done for $r = 2$ in Section 3.2 and for $r = 3$ in Section 5. It is not difficult to change the pseudocode accordingly.)

//伪代码有解释

Theorem 6.1. *The implementation is an $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation of resizable arrays. The amortized cost of grow and shrink operations is $O(r)$. Access operations can be supported in $O(1)$ worst-case time.*

Proof. The correctness of the implementation and the $O(rN^{1/r})$ and $O(N^{1-1/r})$ space bounds follow from the discussion above. The $O(r)$ bound on the amortized cost of grow and shrink operations is proved in Lemma 6.3 below. A way of implementing access operations in $O(1)$ worst-case time is described in the proof of Lemma 6.4 below. $\square$

By letting $r = \log N$, we get the following interesting corollary:

Corollary 6.2. *There is a resizable array implementation that uses $N + O(\log N)$ space to store and array of size N , supporting grow and shrink operations in $O(\log N)$ amortized time and access operations in $O(1)$ worst-case time.*

6.2 Amortized cost of grow and shrink operations

Lemma 6.3. *The amortized cost of grow and shrink operations in the $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation is $O(r)$.*

Proof. We define the cost of a *Grow* or a *Shrink* operation to be the number of item assignments it performs, and obtain an $O(r)$ amortized bound on these costs. Since the total number of operations performed by a *Grow* or a *Shrink* operation is proportional to this defined cost plus one, this is enough to prove the lemma. (The plus one is required to deal with *Shrink* operations that do no item assignments.)

cost指赋值操作
可能会+1
所以指定是cost

The cost of a *Grow* operation is 1, plus the cost of *Combine-Blocks*, if called. Similarly, the cost of a *Shrink* operation is 0, plus the cost of *Split-Blocks*, if called. (Most *Shrink* operations only decrease N and n_0 and do no item assignments.) If a *Combine-Blocks* operation creates a block of size B^k , then its cost is $B(B + B^2 + \dots + B^{k-1}) \leq 2B^k$ (assuming $B \geq 2$). If a *Split-Blocks* operation splits a block of size B^k , then its cost is B^k .

B长度的块复制B次

先分配已credit, 后续cost可用credit支付

An item currently in level i , i.e., in a data block of size B^i , is assigned $r - i$ credit units. Each credit unit can pay for one item assignment. The cost of a *Grow* operation, excluding the cost of *Combine-Blocks*, if called, is 1. The credit assigned to the new item is $r - 1$, since it is always placed at level 1, so the amortized cost of a *Grow* operation, excluding the cost of *Combine-Blocks*, is r .

操作1
+ credits r-1

用credits支付 Combine Blocks即可

The amortized cost of a *Combine-Blocks* operation is 0, since each item assignment moves an item from level i to level $i + 1$, which reduces the credit of the item by 1, paying for the assignment. Hence, the amortized cost of *Grow* is at most r . // credits解释 grow

To handle *Shrink* operations we also assign credits to levels. Each *Shrink* operation adds 3 units of credit to each level. Hence the amortized cost of *Shrink*, excluding the cost of *Split-Blocks*, is at most $3r$.

We next show that the amortized cost of a *Split-Blocks* operation is at most 0. We first note that at least B^k *Shrink* operations must have been performed between two consecutive splits of a block of size B^k . Indeed, following such a split, levels 1 to $k - 1$ contain exactly B^k items. Furthermore, after a *Combine-Blocks* operation that creates a new block of size B^k , levels 1 to $k - 1$ also contain at least B^k items. Thus, there must be at least B^k *Shrink* operations between a *Split-Blocks* operation at level k and the previous *Combine-Blocks* or *Split-Blocks* operation at level k . Similarly, there must be at least B^k *Shrink* operations before the first *Split-Blocks* operation at level k . Therefore, when a *Split-Blocks* occurs at level k , the level has accumulated at least $3B^k$ unused units of credit.

The actual cost of a *Split-Blocks* operation at level k is B^k . The total amount of extra credit that needs to be added to items that are moved from level k to smaller levels is

$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i \leq B^{k+1} \sum_{j \geq 1} jB^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k$,
 第一层B块, 上面每层B-1块, 往下搬需增加k-i credits 保证combine足够支付, 共2B^k, 再加搬运费B^k
 where the last inequality assumes that $B \geq 4$. Thus, the $3B^k$ units of credit of level k are sufficient to cover the cost of the *Split-Blocks* operation and its amortized cost is therefore at most 0. // credits解释 shrink.

Finally, we need to account for the cost of rebuilding. Each *Grow* or *Shrink* operation now adds $2r$ units of credit to the whole data structure. (This, of course, increases the amortized cost of *Grow* and *Shrink* by $2r$.) The number of *Grow* operations between a *Rebuild* operation that doubles B and the previous *Rebuild* operation is at least $B^r - (B/2)^r = N(1 - 2^{-r})N \geq \frac{N}{2}$. The number of *Shrink* operation between a *Rebuild* operation that halves B and the previous rebuild operation is at least $(B/2)^r - (B/4)^r = (2^r - 1)N \geq N$. Thus, when a *Rebuild* is about to take place, the data structure has accumulated at least rN unused credit units. The actual cost of a *Rebuild* is exactly N . The total amount of credit that needs to be assigned to items is at most $(r-1)N$. Thus, the amortized cost of *Rebuild* is 0. // credits解释全局

The amortized analysis of *Rebuild* given above is very loose. When a *Rebuild* doubles B , the total credit of all items is actually decreased. Thus, each *Grow* operation needs to add only $1/(1-2^{-r}) \leq 2$ units of credit to the data structure. Similarly, each *Shrink* needs to add only $r/(2^r - 1) \leq 1$ units of credit to the data structure. This, of course, does not change the $O(r)$ amortized cost of *Grow* and *Shrink* operations.

The analysis above implicitly assumes that r is a constant, i.e., that it does not vary with N . It is easy to adapt the analysis to the case of non-constant $r = r(N)$.

6.3 Accessing and modifying items in $O(1)$ worst-case time

In this section we show that the $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation of Section 6.1 can support access and modify operations in $O(1)$ worst-case time. We assume that the index of the most significant set bit in a machine word x , which we denote by $msb(x)$, can be determined in $O(1)$ time. (More on this assumption later.)

Lemma 6.4. *The $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation can support access and modify operations in $O(1)$ worst-case time.*

To simplify the efficient implementation of access operations we make one small changes to the data structure. We let n_1 be the number of full data blocks of size B , i.e., not including the partially filled block of size B which contains $n_0 < B$ items, if there is one. The numbers $n_2, \dots, n_{r-1}$ are still the number of full blocks of sizes $B^2, \dots, B^{r-1}$. (All blocks of these sizes are full.) We also assume that B is a power of 2, i.e., $B = 2^b$ for some integer $b > 0$. // 更改 n_i 定义

Note that $N = \sum_{j=0}^{r-1} n_j B^j = (n_{r-1}, \dots, n_1, n_0)_B = \sum_{j=0}^{r-1} n_j B^j$, where $0 \leq n_j \leq 2B$, for $0 \leq j < r$. Thus, $(n_{r-1}, \dots, n_1, n_0)$ is a possibly redundant base- B representation of N . We store $n_{r-1}, \dots, n_1, n_0$ compactly in two words $N^0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B$ and $N^1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B$, where $n_j = n_j^0 + B n_j^1$, $0 \leq n_j^0 < B$, $0 \leq n_j^1 \leq 2$, for $0 \leq j < r$. Note that N^a , for $a = 0, 1$, is the concatenation the b -bit representations of each one of $n_{r-1}^a, \dots, n_1^a, n_0^a$. Each n_j^a can be extracted from N^a in constant time using simple logical and shift operations. The data structure can easily maintain N^0 and N^1 in addition to N . // 定义 N^0, N^1

We let $N_k = \sum_{j=0}^{k-1} n_j B^j$, for $0 \leq k < r$. (We have $N_r = N$ and $N_0 = 0$.) Note that N_k is the total number of items in blocks of size at most B^{k-1} and thus $N - N_k$ is the number of items stored in blocks of size at least B^k . Recall that larger blocks contain items with lower index. Thus, the i -th item resides in a block of size at most B^k only if $N - N_{k+1} \leq i$. (Recall that indices start from 0.)

The numbers N_k , for $0 \leq k < r$ can be explicitly maintained by the data structure, without affecting the asymptotic running times of the various operations. Alternatively $N_k = (N^0 \wedge M_k) + B(N^1 \wedge M_k)$, where $\wedge$ is the bitwise logical and operation and M_k is a mask of kb 1's, i.e., $M_k = 2^{kb} - 1 = (1 \ll kb) - 1$, where $\ll$ denotes left shift. (The masks can be precomputed.) The multiplication by B can also be replaced by a left shift of size b , i.e., $N^k = (N^0 \wedge M_k) + (N^1 \wedge M_k) \ll b$. $B = 2^b$

To access the i -th item in the resizable array, where $0 \leq i < N$, we need to find the unique $0 \leq k < r$ such that $N - N_{k+1} \leq i < N - N_k$. The i -th item is then in position $(i - (N - N_{k+1})) \bmod B^k$ of the $\lfloor \frac{i - (N - N_{k+1})}{B^k} \rfloor$ -th block of size B^k . (If $k = 0$ then the i -th item is the $(i - (N - N_1))$ -th item in the single partially filled block of size B .) Since B^k is a power of 2, the i -th item can be easily accessed in constant time, using logical and shift operations, once k is known. (Note that $x \bmod B^k = x \wedge M_k$ and $\lfloor x/B^k \rfloor = x \gg kb$, where $\gg$ denotes a right shift.) // 正向寻找

Let $x = (N - 1) - i$ be the index of the i -th item from the end of the array, again starting the numbering from 0. We then need to find the unique $0 \leq k < r$ such that $N_k \leq x < N_{k+1}$.

Let $x = (x_{r-1}, \dots, x_1, x_0)_B$ be the standard base- B representation of x . To find k , we begin by finding the largest index ℓ such that $x_\ell > 0$. This can be easily done by finding the index of the most significant set bit in x , i.e., $\ell \leftarrow \lfloor \text{msb}(x)/b \rfloor$. (We assume again that indices start from 0.)

Since $x_\ell > 0$ we have $x \geq B^\ell$. Also $N_{\ell-1} \leq 2B \sum_{j=0}^{\ell-2} B^j = \frac{2B}{B-1} (B^{\ell-1} - 1) \leq 3B^{\ell-1}$, assuming $B \geq 4$. Thus, $N_{\ell-1} \leq x$ and therefore $k \geq \ell - 1$. If $x < N_\ell$ then $k = \ell - 1$. Otherwise, if $N_\ell \leq x < N_{\ell+1}$ then of course $k = \ell$. If $N_{\ell+1} \leq x$, then $k = \ell'$ where $\ell' > \ell$ is the smallest index for which $n_{\ell'} > 0$. (Such an index must exist as $x < N$.) This is equivalent to finding the smallest $\ell' > \ell$ for which either $n_{\ell'}^0 > 0$ or $n_{\ell'}^1 > 0$. This can be easily done as follows: $\ell' \leftarrow \lfloor \text{lsb}((N^0 \vee N^1) \wedge (\neg M_{\ell+1})) / b \rfloor$, where $\text{lsb}(x)$ is the index of the least significant set bit in x , $\vee$ is the logical or operation and $\neg$ denotes complementation.

If x is a number, then $x \wedge (-x)$ leaves only the rightmost 1 in x . (See Equation (37) in Knuth [12].) Thus, lsb can be easily reduced to msb , namely $\text{lsb}(x) = \text{msb}(x \wedge (-x))$. (No reduction in the other direction is known.) // 例: 通过 msb 快速定位 lsb

Computing the index of most significant set bit in of a number x is equivalent to computing the *binary logarithm*, i.e., $\text{msb}(x) = \lfloor \lg x \rfloor$. Several ways of finding the index of the most significant set bit in a word in constant time are discussed in Knuth [12]. One option, if available, is to convert x to floating point and extract the exponent. Fredman and Willard [7] describe a clever way of computing $\text{msb}(x)$ using a constant number of logical, shift and arithmetical operations.

Their method requires the use of multiplication. The use of multiplication is essential. Brodal, see Knuth [12], devised a way of computing $msb(x)$ using standard logical, arithmetical and shift operations, without using multiplication, in $O(\log \log w)$ time, where w is the number of bits in a machine word. A matching lower bound was obtained by Brodnik et al. [3].) // 先前研究表明 msb 可在常数时间找到

应用

Obtaining a constant access time independent of r is only of theoretical interest as in practice the value of r used is expected to be very small. The simple $O(\log r)$ binary search algorithm, or even the naïve $O(r)$ algorithm, for locating the value of k would probably work best.

7 A data structuring transformation

If r is a constant, then the $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation of the previous section is optimal, both in the amount of temporary storage used and in its $O(r) = O(1)$ amortized cost of grow and shrink operations. If $r = r(N)$ is a (slowly) growing function of N , it is interesting to ask whether the $O(rN^{1/r})$ extra space needed for storing the array can be reduced to $O(N^{1/r})$, and whether the amortized cost of grow and shrink operations must be $\Omega(r)$. We may assume that $r(N) = O(\log N)$, since $N^{1/O(\log N)} = O(1)$.

We show here that the amount of extra space can be reduced to $O(N^{1/r})$, at least if $r(N) \leq \frac{1}{2} \frac{\log N}{\log \log N}$. In Section 9 we show that the amortized cost of grow and shrink operations must be $\Omega(r)$.

We reduce the extra space needed from $O(rN^{1/r})$ to $O(N^{1/r})$ using a surprisingly simple transformation. To state the transformation, we need the following definition:

Definition 7.1 (Standard $(s(N), t(N))$ -implementation). *An $(s(N), t(N))$ -implementation is standard if it satisfies the following conditions: (i) When a new block B is allocated, items from some other blocks $A_1, A_2, \dots, A_k$ are immediately copied into it in sequential order. More specifically, first all the items of A_1 are copied into the first positions of B and then A_1 is released, then the items of A_2 are copied to the next positions of B , and A_2 is released, and so on. (ii) Following each such operation, the total space used by the data structure is $N + s(N)$.*

In other words, an $(s(N), t(N))$ -implementation is standard if the $t(N)$ temporary extra space is only briefly needed following the allocation of new blocks. It is not difficult to check that all implementations given in the paper are standard. We can now obtain the following result.

Lemma 7.2. *A standard $(s(N), O(N))$ -implementation can be converted into an $(s(N) + O(\frac{N}{t(N)}), s(N) + O(t(N)))$ -implementation, for any non-decreasing function $t(N)$, without increasing the asymptotic worst-case cost of access operations and the asymptotic amortized cost of grow and shrink operations.*

Proof. Given the definition of standard implementations, we only need to consider the allocation of a new block B that is to receive the items currently stored in $A_1, A_2, \dots, A_k$. Let b be the length of B . If $b \leq t(N)$, there is no problem. Otherwise, instead of allocating B , we sequentially allocate and fill $\lceil b/t(N) \rceil$ blocks $B_1, B_2, \dots, B_{\lceil b/t(N) \rceil}$ of size $t(N)$. We of course need to keep pointers to these blocks. Since the total size of all blocks allocated at any given time is at most $N + s(N)$, the number of extra pointers needed is at most $(N + s(N))/t(N)$. The total space needed to store an array of size N is at most $N + s(N) + O(\frac{N}{t(N)})$. The total space needed at any time by the data structure is at most $N + s(N) + O(\frac{N}{t(N)}) + t(N)$, which is $s(N) + O(t(N))$ since we may assume that $t(N) \geq \sqrt{N}$. 即新块分成多个小块申请, 降低 $t(N)$ 因更多指针提高 $s(N)$ $\square$

Using the lemma, we easily get the following theorem:

Theorem 7.3. For any integer $r = r(N) \leq \frac{1}{2} \frac{\log N}{\log \log N}$, there exists an $(O(N^{1/r}), O(N^{1-1/r}))$ -implementation that supports access and modify operations in $O(1)$ worst-case time and grow and shrink operations in $O(r)$ amortized time.

Proof. Apply the transformation of Lemma 7.2, with $t(N) = N^{1-1/r}$, to the standard $(O(rN^{1/(2r)}), O(N^{1-1/(2r)}))$ -implementation of Section 6. The result is an $(O(rN^{1/(2r)} + N^{1/r}), O(N^{1-1/r}))$ -implementation. If $r \leq \frac{1}{2} \frac{\log N}{\log \log N}$ then $2r \log r \leq \log n$ and hence $rN^{1/(2r)} \leq N^{1/r}$. Thus the obtained implementation satisfies the requirements of the theorem. 代入参数即可 $\square$

8 The growth game

To get an indication as to how a truly optimal data structure should handle grow operations, and to obtain an $\Omega(r)$ lower bound on the amortized cost of the grow operations of a standard data structure that is only allowed $N + O(N^{1/r})$ space to store an array of size N , we define and analyze an interesting solitaire, i.e., a one-player, game called the *growth game*, which is an abstraction of the way standard data structures can handle grow operations. (See Definition 7.1.)

8.1 The (N, k, ℓ) -growth game.

Let N, k and ℓ be fixed parameters. We are required to insert N items, one by one, into k initially empty subarrays $A_1, A_2, \dots, A_k$ that represent a resizable array A . The first items of A are stored in A_k , the next in A_{k-1} , and so on. (This apparent reversal turns out to simplify things.) Let $a_1, a_2, \dots, a_k$ be the number of items in $A_1, A_2, \dots, A_k$, respectively. // 定义 growth game -

Some of the subarrays $A_1, A_2, \dots, A_k$ may be *empty*, i.e., contain no items. (If A_i is empty then $a_i = 0$.) Empty subarrays, if any, are assumed to be the first subarrays, i.e., if $a_i = 0$ then $a_1 = a_2 = \dots = a_i = 0$. The first nonempty subarray may contain up to ℓ *vacant* positions. All other subarrays are *full*, i.e., the number of items stored in them is equal to the size of the memory block allocated for them. The extra space used to store the items is thus always at most $k + \ell$ k 个空位

A grow operation, which adds a new item to the array, is implemented as follows. If the first non-empty subarray has vacant positions, the item is simply placed in its first vacant position. (See Figure 6(a).) If all non-empty subarrays are full, but there is at least one empty subarray, a new subarray of size $\ell + 1$ is allocated and the new item is placed in its first position. The remaining ℓ positions are left vacant. This subarray replaces the last empty subarray. (See Figure 6(b).) The cost of inserting the new item in both these cases is defined to be 1. (For simplicity, we ignore the cost of allocation.) Note that in these first two cases the player of the game has no real choice. // 插入操作

The game is more interesting when all k subarrays are full. We now have k possible moves. For each $i \in [k] = \{1, 2, \dots, k\}$, we can allocate a new subarray of size $(\ell + 1) + \sum_{j=1}^i a_j$, copy the items in $A_i, A_{i-1}, \dots, A_1$, in this order, to the new subarray, and then place the new item in the first vacant position of the new subarray. (The new subarray will still have ℓ vacant positions.) The old subarrays $A_1, \dots, A_i$ are deallocated. The new subarray becomes A_i while $A_1, A_2, \dots, A_{i-1}$ become empty. (See Figure 6(c).) The cost of the operation is defined to be $1 + \sum_{j=1}^i a_j$. (Again we count only item assignments and ignore the cost of allocations and deallocations.) A move of the second type is actually a move of the third type, but i is forced to be the largest index for which $a_i = 0$. // 全满

Definition 8.1 (Total and amortized cost). Let $C_{N,k,\ell}$ be the minimum total cost required to play the (N, k, ℓ) -growth game. Let $A_{N,k,\ell} = \frac{C_{N,k,\ell}}{N}$ be the corresponding amortized cost of a single grow operation. 11 部分

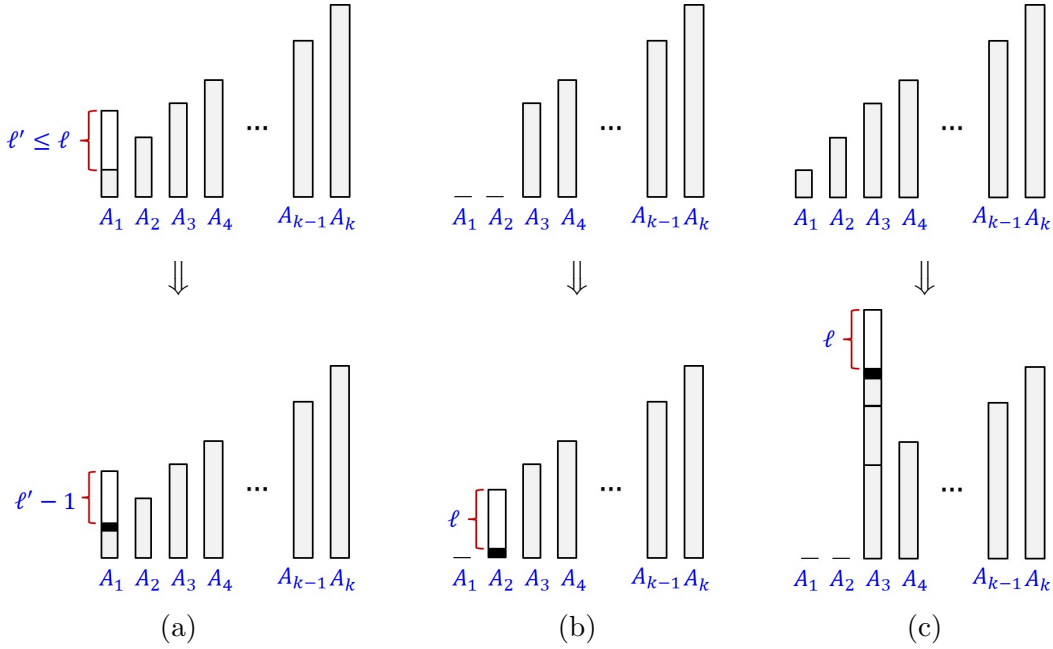


Figure 6: Possible moves in the growth game. (a) If the first non-empty subarray has vacant positions, a new item is inserted into its first vacant position. The cost of the operation is 1. (b) If there are no vacant positions, but there is at least one empty subarray, then a new array of size $\ell + 1$ is allocated. (c) If there are no vacant positions, the subarrays $A_1, A_2, \dots, A_i$, for some $i \in [k]$, are merged into a newly formed A_i , initially with $\ell + 1$ vacant positions. The new item is placed in the first vacant position, leaving ℓ vacant positions. The subarrays $A_1, A_2, \dots, A_{i-1}$ become empty. The cost of the operation is $1 + \sum_{j=1}^i a_j$. (In the example, $i = 3$.)

We are interested in finding $C_{N,k,\ell}$, and hence $A_{N,k,\ell}$, and an optimal sequence of moves that achieves these values. The solution to these problems turns out to be fairly simple, and also fairly interesting. The values of $A_{N,cN^{1/r},cN^{1/r}}$, for some constant $c > 0$, can be used, as we shall see, to bound from below the amortized cost of a grow operation when only $cN^{1/r}$ extra space is allowed. We are more liberal here, since we assume that N is known in advance and we allow $cN^{1/r}$ extra space even when the array contains many fewer than N items. We also do not impose any bound on the amount of space used temporarily during grow operations. All this only makes our lower bound stronger.

8.2 Reduction to the case $\ell = 0$.

growth game 抽象中所有多出的
已知条件都只让 lower bound 更强

It might seem that allowing up to ℓ vacant positions in one of the subarrays is an important feature of the game that helps reduce the amortized cost of growth operations. We begin by showing, perhaps surprisingly, that vacant positions do not help much, at least for most values of the parameters. More specifically, we describe a reduction from the $\ell > 0$ case to the $\ell = 0$ case.

Lemma 8.2. *For every N divisible by $\ell + 1$, $C_{N,k,\ell} = (\ell + 1)C_{\frac{N}{\ell+1},k,0}$ and $A_{N,k,\ell} = A_{\frac{N}{\ell+1},k,0}$.*

证明 $\ell=0$ 不太

Proof. Playing the (N, k, ℓ) -growth game amounts to playing the $(\frac{N}{\ell+1}, k, 0)$ -growth game on blocks of $\ell + 1$ items. To see this, note that when a grow operation in the (N, k, ℓ) -growth game leaves ℓ vacant positions, the next ℓ grow operations must immediately fill these positions. $\square$

影响结果